Database Documentation

AI Notice: Fed AI project & database context including queries in order to help build out the documentation quicker while still making sure proper information is documented and explained.
Note: All SQL queries for table creation and seed data can be found in the repository at client_site/db/init.sql.

Database name: emmanue0_RocketRentals
The database uses four tables organized in a hierarchical chain to model the client's tool rental inventory. The relationship flows from broad groupings down to individual physical items:
tool_categories → tool_types → tools → tool_units

Table 1: tool_categories

Purpose: Stores the broad rental categories that organize the entire tool catalog on the site.
What data it stores: Each row represents one top-level category such as "Power Tools," "Lawn & Outdoor," or "Cleaning." It holds the display name, a URL-safe key for linking/filtering, a short description, whether the category is featured, a display ordering value, and an optional image URL for the category card.
How it supports the client site: This table drives the category sidebar navigation on the tools pages (toolsCategories.php, tools.php) and the category card grid on the landing page. The key column is used as the ?category= URL parameter for filtering. The featured flag controls which categories display a "Most Popular" pill.
How it relates to other tables: One-to-many with tool_types, each category contains one or more tool types. This is the top of the hierarchy; it has no foreign keys itself.

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
key	VARCHAR(30)	NOT NULL, UNIQUE	URL-safe slug (e.g. power, lawn)
name	VARCHAR(100)	NOT NULL	Display name (e.g. "Power Tools")
description	VARCHAR(255)	NOT NULL	Short blurb shown on the category card

image_url	VARCHAR(500)	DEFAULT NULL	Category card image; NULL for now
featured	TINYINT(1)	NOT NULL, DEFAULT 0	Whether to show "Most Popular" pill
sort_order	INT	NOT NULL, DEFAULT 0	Controls display order in sidebar/grid
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Row creation timestamp
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp

Primary Key: id Foreign Keys: None

Table 2: tool_types

Purpose: Stores sub-groupings within each category, providing a classification layer between the broad category and individual tools.
What data it stores: Each row represents a type of tool within a category - for example, under "Power Tools" there are types like "Drills & Drivers," "Saws," and "Hammers." It stores a display name, a URL-safe key, an optional description, and a display ordering value.
How it supports the client site: This table allows the catalog to scale. As the client adds more tools, types enable sub-filtering and section headers within a category page (e.g. browsing just "Saws" within Power Tools) without restructuring the database. Currently the site filters at the category level, but this table is ready for future sub-navigation.
How it relates to other tables: Many-to-one with tool_categories (each type belongs to exactly one category via category_id). One-to-many with tools (each type contains one or more specific tool products).

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
category_id	INT	NOT NULL, FOREIGN KEY	References tool_categories.id
key	VARCHAR(50)	NOT NULL	URL-safe slug (e.g. drills, saws)

name	VARCHAR(100)	NOT NULL	Display name (e.g. "Drills & Drivers")
description	VARCHAR(255)	DEFAULT NULL	Optional blurb for sub-filtering pages
sort_order	INT	NOT NULL, DEFAULT 0	Controls display order within category
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Row creation timestamp
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp

Primary Key: id Foreign Keys: category_id → tool_categories(id) ON DELETE CASCADE
Unique Constraint: (category_id, key) - no duplicate type slugs within the same category

---

Table 3: tools

Purpose: Stores individual rentable products - each row is one tool listing that appears as a card on the website.
What data it stores: The tool's display name, a URL-safe slug, its description, daily rental rate, required deposit amount, whether it requires delivery (versus customer pickup), an optional product image URL, and whether the listing is currently active on the site. This represents the model of a tool (e.g. "Circular Saw"), not a specific physical unit.
How it supports the client site: This table directly maps to the RentalTool PHP class and populates the tool cards on tools.php. Each card displays the tool name, description, formatted daily rate, and a rental note (standard rental vs. delivery-only). The slug column supports clean URLs for future individual tool detail pages. The is_active flag allows the client to hide tools without deleting data.
How it relates to other tables: Many-to-one with tool_types (each tool belongs to exactly one type via type_id, which in turn belongs to a category). One-to-many with tool_units (each tool product can have multiple physical units in inventory).

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
type_id	INT	NOT NULL, FOREIGN KEY	References tool_types.id

name	VARCHAR(150)	NOT NULL	Display name (e.g. "Impact Driver")
slug	VARCHAR(150)	NOT NULL, UNIQUE	URL-safe identifier (e.g. impact-driver)
description	TEXT	NOT NULL	Tool card description text
daily_rate	DECIMAL(8, 2)	NOT NULL	Rental price per day (e.g. 18.00)
deposit	DECIMAL(8, 2)	NOT NULL, DEFAULT 0.00	Required deposit amount
delivery_only	TINYINT(1)	NOT NULL, DEFAULT 0	1 = delivery + pickup only, 0 = standard rental
image_url	VARCHAR(500)	DEFAULT NULL	Product photo URL; NULL for now
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	1 = visible on site, 0 = hidden
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Row creation timestamp
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp

Primary Key: id Foreign Keys: type_id → tool_types(id) ON DELETE CASCADE

Table 4: tool_units

Purpose: Tracks individual physical inventory items - each row is one real piece of equipment that Rocket Rentals owns and rents out.
What data it stores: Each row represents a specific physical unit of a tool product, identified by a serial number (e.g. RR-PWR-001). It stores the unit's current condition (new, good, fair, or in maintenance), its rental status (available, rented, reserved, or retired), optional internal notes, and the date the unit was acquired.
How it supports the client site: This is the table that makes real rental operations possible. When a customer rents a "Circular Saw," a specific unit of that saw is reserved. The status column drives availability - the site can count available units per tool to show real-time stock. The condition column helps the client decide which units need maintenance or replacement. This table will connect to future reservation/order tables.

How it relates to other tables: Many-to-one with tools (each unit belongs to exactly one tool product via tool_id). Multiple units can exist for the same tool - for example, Rocket Rentals might own three Impact Drivers. This is the bottom of the hierarchy; no other tables reference it yet (future reservation tables will).

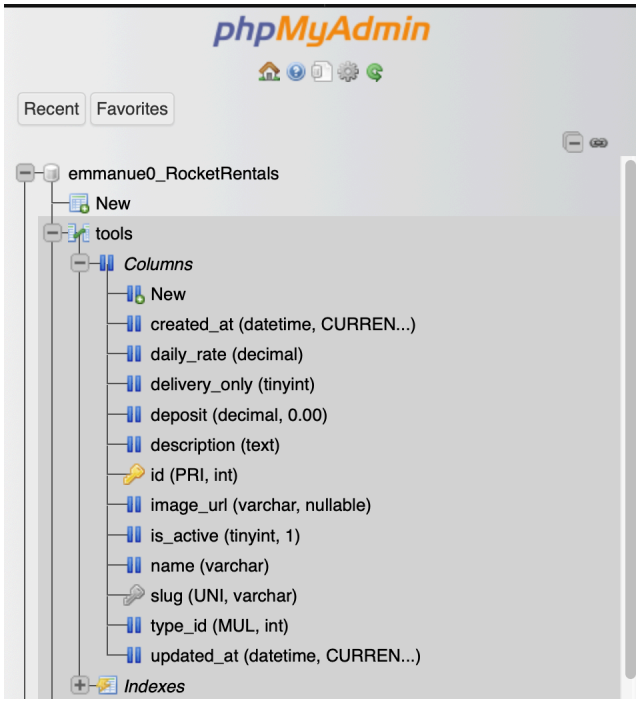
Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
tool_id	INT	NOT NULL, FOREIGN KEY	References tools.id
serial_number	VARCHAR(100)	DEFAULT NULL, UNIQUE	Physical asset identifier (e.g. RR-PWR-001)
condition	ENUM('new','good','fair','maintenance')	NOT NULL, DEFAULT 'good'	Physical condition of the unit
status	ENUM('available','rented','reserved','retired')	NOT NULL, DEFAULT 'available'	Current rental availability
notes	TEXT	DEFAULT NULL	Internal notes (e.g. "scratched handle")
acquired_date	DATE	DEFAULT NULL	Date this unit was acquired
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Row creation timestamp
updated_at	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp

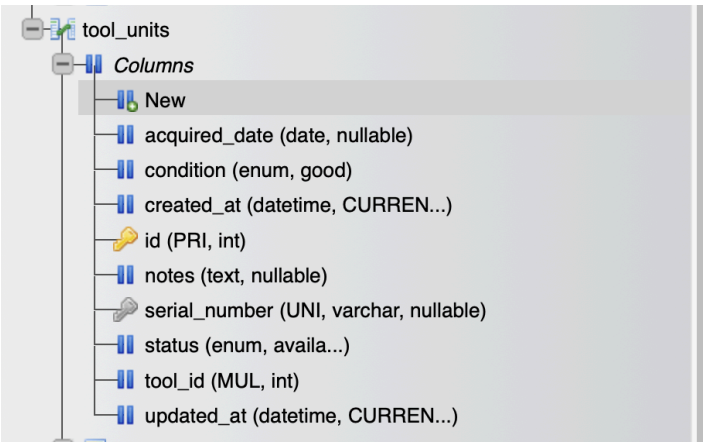
Primary Key: id Foreign Keys: tool_id → tools(id) ON DELETE CASCADE

Relationship Summary

```
tool_categories (6 rows)
└─── tool_types (16 rows)       FK: category_id → tool_categories.id
     └─── tools (18 rows)      FK: type_id → tool_types.id
          └─── tool_units (46 rows)  FK: tool_id → tools.id
```

All foreign keys use ON DELETE CASCADE - deleting a category removes its types, their tools, and those tools' units. All tables use InnoDB for foreign key support and utf8mb4 character encoding.





Client Feedback & Approval

Demo Presentation & Feedback Review

Date and Time: Friday, April 3, 2026 at 5:00 PM PST
Meeting Format: In person at a client's home

Summary of Discussion

We met to review the current state of the Rocket Rentals website, walk through peer feedback received from class, and discuss the newly implemented database-driven tool management system.

Site Walkthrough

I guided the client through the full customer-facing experience:

- Homepage with category quick-links and search
- Tool Categories page (toolsCategories.php) — now loading categories dynamically from the database with real tool names in the sample lists instead of hardcoded placeholder text
- Tools listing page (tools.php) — filtering by category via URL query strings, with sidebar navigation and tool cards pulling live data from the MySQL database
- Checkout flow — the existing multi-step form with schedule, delivery address (Google Places autocomplete), and review steps
- Admin dashboard (/admin/) — the new internal tool management interface for adding, editing, and deleting tools from the inventory

Peer Feedback Review - Major Themes Identified

1. Navigation clarity (High Priority): Multiple peers noted that the relationship between the homepage, category pages, and individual tool listings could be clearer. Some found it confusing that static HTML category pages existed alongside the PHP-driven pages.
2. Placeholder content (High Priority): Several peers pointed out that text like "Placeholder" badges, "placeholder inventory," and deposit values showing "XX" made the site feel unfinished and undermined credibility.
3. Visual consistency (Medium Priority): Feedback mentioned that the checkout page styling felt slightly different from the tools/categories pages. The overall orange/white Rocket Rentals branding was well-received, but consistency across all pages was suggested.
4. Missing tool images (Medium Priority): Peers noted that tool cards have no images, which makes it harder for customers to identify tools at a glance. Comparison to competitor sites like Home Depot and Sunbelt Rentals was made.

5. Mobile responsiveness (Low Priority): A few peers tested on mobile and noted that while the layout adjusts, some elements like the sidebar could be improved on smaller screens.

Feedback the Client Agreed to Address

- Remove all "Placeholder" language and badges. The client agreed this was the highest priority. Since we now have a real database connection, all placeholder text has been replaced with "Live" badges and real inventory language. Status: Implemented.
- Connect the site to a real database. The client specifically requested that the tool catalog be backed by a real MySQL database rather than hardcoded PHP arrays, so that inventory changes don't require code edits. This was the primary focus of our latest development sprint. Status: Implemented.
- Build an internal admin interface for tool management. The client expressed that he needs the ability to add new tools, update pricing (daily rates and deposits), and remove discontinued tools without contacting the developer. We built a simple admin dashboard at /admin/ that is not linked from the customer site and is accessed via direct URL only. The client reviewed the add, edit, and delete workflows and confirmed they meet his current needs. Status: Implemented.
- Add image support to the database schema. The client wants product photos on tool cards similar to Home Depot's listings. We added image_url columns to both the tool_categories and tools database tables. These are NULL for now but ready to accept URLs or future S3 bucket paths once the client provides product photography. Status: Schema ready, images pending.
- Fill in real deposit amounts. The client provided guidance on deposit pricing (roughly 2.5–3x the daily rate), and we populated the database with real values instead of the "XX" placeholders. Status: Implemented.

Peer Feedback the Client Chose Not to Incorporate

- Adding user accounts / login for customers. Some peers suggested customer accounts for order history. The client decided this is premature — the business model currently handles reservations as quotes with payment collected at delivery. Customer accounts will be revisited in a future phase.
- Online payment processing. A peer suggested integrating Stripe or PayPal. The client prefers to keep payment offline for now, as the business handles transactions at the point of delivery. This may change as the business scales.

Additional Feedback and Concerns Raised by the Client

- The client emphasized that the admin interface does not need authentication for now since only he and I know the URL, but acknowledged that securing it with login credentials is important before any public launch.

- The client asked about the possibility of managing tool units (individual physical items with serial numbers) through the admin interface in addition to tool models. Currently the admin manages tool products only; unit management is a planned future feature.
- The client expressed satisfaction with the category organization (Power Tools, Lawn & Outdoor, Concrete & Masonry, Cleaning, Ladders & Lifts, Trailers & Hauling) and confirmed no new categories are needed at this time.

List of Changes / Refinements Made Based on Discussion

1. Replaced all hardcoded PHP tool and category arrays with live database queries via PDO
2. Created includes/db.php as a shared database connection file
3. Updated toolsCategories.php to pull categories and sample tool names from the database
4. Updated tools.php to use prepared statements for category filtering via query strings
5. Built admin/index.php dashboard with full tool listing and action buttons
6. Built admin/add-tool.php for inserting new tools (with validation and sanitization)
7. Built admin/edit-tool.php for updating existing tools (loaded via query string, prepared statements)
8. Built admin/delete-tool.php for removing tools (POST-only, with confirmation)
9. Replaced all "Placeholder" badges and copy with "Live" indicators and real inventory language
10. Populated real deposit values across all 18 tools in the database

---

Client Approval

RR Website Meeting External Inbox ×

Alexander Sosa <rocketrentals.ri@gmail.com> 3:22 PM (4 hours ago)
to me

Hi Emmanuel,
I've reviewed the latest version of the website. The management features for tools work as discussed, and the customer-facing pages now display real inventory from the database. I approve these changes and am happy with the direction of the project.

Thanks,
Alex Sosa

Reflection on Feedback & Future Implementation

Most Significant Changes

The most significant change was moving from hardcoded PHP arrays to a live MySQL database for all tool and category data. Previously, tools.php contained 18 manually written new

RentalTool(...) calls and toolsCategories.php had 6 hardcoded new ToolCategory(...) blocks. Any change to pricing, descriptions, or the catalog required editing PHP source code.
Before: Tool data was hardcoded in PHP with placeholder deposits ("XX") and "Placeholder" badges throughout the UI.
After: All data comes from the emmanue0_RocketRentals database via PDO queries with prepared statements. The admin dashboard allows the client to manage the catalog independently. Sidebar navigation and category listings are dynamically generated from the database, so adding a new category or tool type automatically updates every page.

How Feedback Improved User Experience

Peer feedback directly led to removing placeholder language that undermined the site's professionalism. The database integration addressed the root cause — data is now real and maintainable. Client feedback drove the admin dashboard, which aligns with the business goal of operational independence (the client shouldn't need a developer to change a price).

Suggestions Not Implemented

Customer accounts and online payment were not implemented because the client's current business model handles both offline. These are intentional deferrals, not oversights.

Features Still to Be Implemented (Roadmap)

1. Tool images — image_url columns exist in the database but are currently NULL. Pending product photography or stock images from the client.
2. Admin authentication — Login system to secure the /admin/ section before public deployment.
3. Tool unit management — Admin interface for managing individual physical units (serial numbers, condition, availability status) within the existing tool_units table.
4. Customer accounts and reservation system — User registration, cart persistence in the database, and real reservation tracking tied to tool unit availability.
5. Checkout integration with database — Connect the existing checkout form to create actual reservation records in the database instead of the current session-only mock flow.
6. Mobile sidebar improvements — Collapsible category sidebar on smaller screens for better mobile UX.